

STOP REVIEWING YOUR AGENT'S FLAKY PLAYWRIGHT TESTS

Turn ESLint into the always-on reviewer your agents can't ignore



John Hill · @unlikelyzero · maintainer, eslint-plugin-playwright

in collaboration with **ThinkSys**

THINKSYS × 

IN THE NEXT 45 MINUTES

WHAT YOU'LL LEAVE WITH

1. **The problem**
   why agent-written Playwright tests flake, and why review doesn't catch it
2. **The mechanism**
   an ESLint hook that lands the fix instruction at the moment of the mistake
3. **The examples**
   7 rules: the shortcut the agent took, and the test after the lint hook
4. **Rolling it out**
   turn every rule on today without fixing anything today

We'll pause for questions after rule 2 and at the end — drop them in the Q&A panel as they come.

THINKSYS ×

AGENTS WRITE TESTS FASTER THAN YOU CAN REVIEW THEM

Ask any agent to make a failing test pass. It reaches for whatever works *right now*.

```
1 await page.waitForTimeout(2000);
2 await page.locator('form.composer > div > div > button').click();
3 await page.locator('button.send').first().click();
4 await button.click({ force: true });
5 if (await banner.isVisible()) await banner.click();
```



Green today. Three weeks later it's the flake blocking someone else's PR.

PROMPTS ARE SUGGESTIONS. LINT IS A VERDICT.

AGENTS.MD / .CURSORRULES

- Diluted as context grows
- Agents route around them

ESLINT RULE

- Same verdict every time
- Error lands *at the moment of the mistake*

Keep both — but make the prompt *point at* the verdict:

```
# AGENTS.md
Run `npm run lint` before you report a test as done. Every error is
a rule, and every rule has a doc — read it, don't suppress it:
https://github.com/playwright-community/eslint-plugin-playwright/tree/main/docs/rules
```

THE MECHANISM · 1 OF 2

WIRE IT IN BEFORE A BROWSER EVER LAUNCHES

```
// .claude/settings.json
{ "hooks": { "PostToolUse": [{
    "matcher": "Edit|Write|MultiEdit",
    "hooks": [{ "type": "command",
                "command": "bash $CLAUDE_PROJECT_DIR/scripts/lint-hook.sh" }]
}]}}
```

Exit 2 + stderr → the fix instruction becomes the result of the agent's own edit.
Cursor: `hooks.json → afterFileEdit` · Copilot: pre-commit · Playwright MCP: same script in the tool wrapper.

THE MECHANISM · 2 OF 2

REWRITE MESSAGES AS FIX INSTRUCTIONS

STOCK

> *Unexpected use of page.waitForTimeout().*

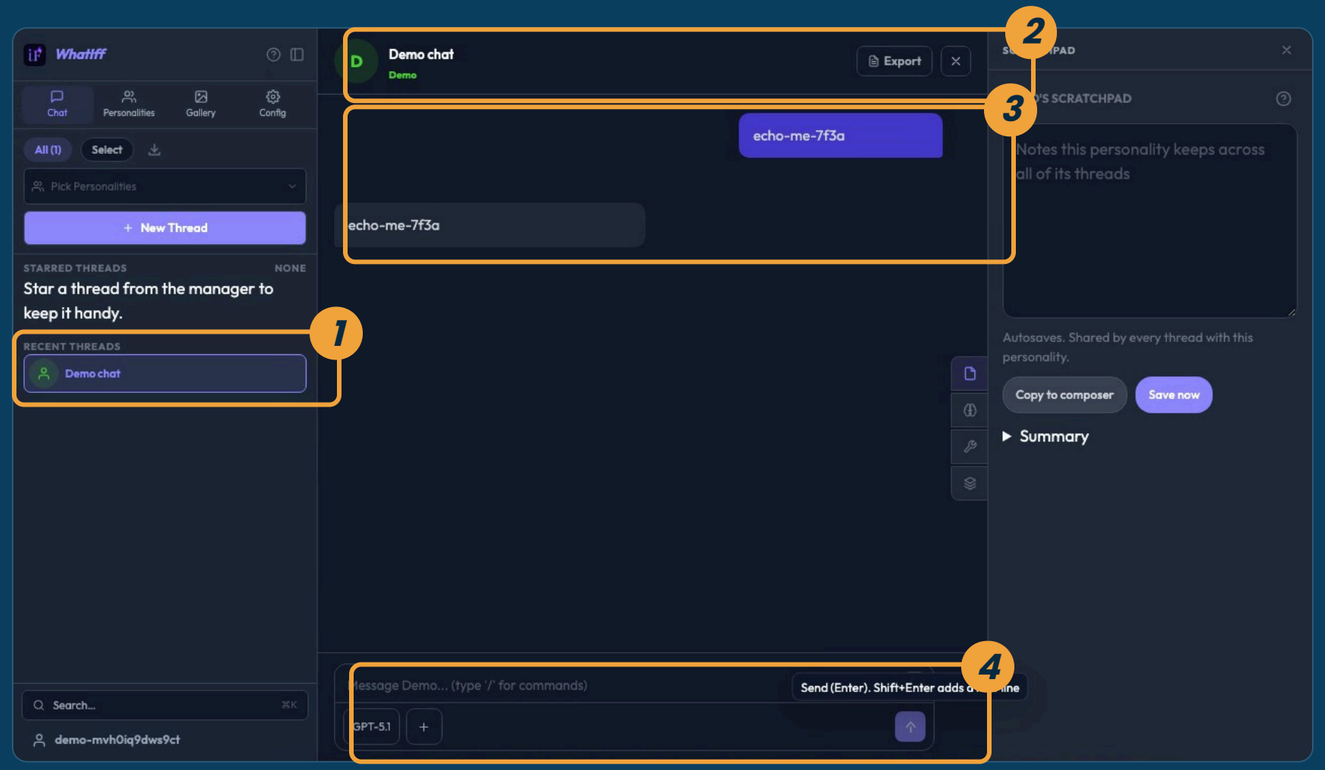
REWRITTEN

> *Remove page.waitForTimeout(). Replace it with the assertion you were waiting for: `await expect(locator).toBeVisible()`…*
> *If nothing observable changes, there is nothing to wait for.*

Same rule. One repairs on the first try. **Where** the verdict lands + **what** it says = the "linted" setup in every demo that follows.

THE APP IN THE VIDEOS

MEET WHAT-IFF



1. Recent threads
2. Thread name in the header
3. Your message, and the assistant's reply
4. Composer: `Message box`, `Send message button`

- Open-source AI chat app: Angular front end, Go API
- Mock-LLM mode: the assistant echoes what you send, so replies are predictable
- Each video: fresh account, seeded chat, type, send, check the reply
- Each runs under one injected *flake seam* (a slow API, a late banner, renamed CSS), named on the rule's first slide
- left the agent's first draft · right after the lint hook

RULE 1 / 7

no-wait-for-timeout

Prompt: *Write a test that sends a chat message and checks the assistant replies.*

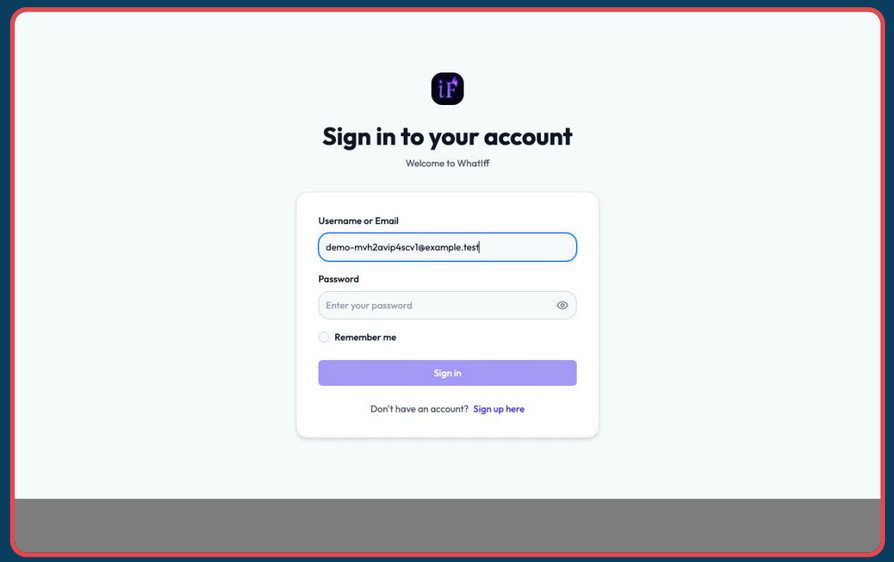
Agent's shortcut: `waitForTimeout(2000)`

Flake seam: `slowApi`

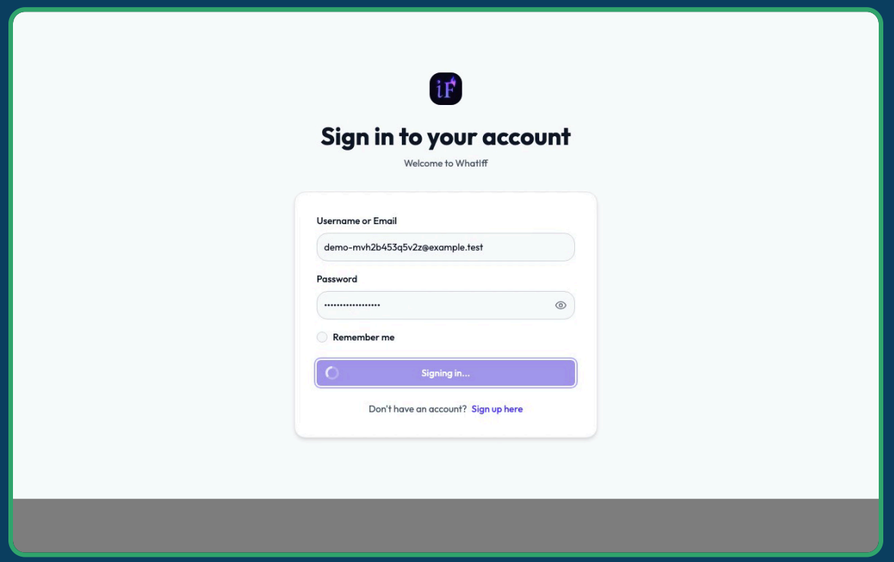
Probably: network latency — the reply isn't back in 2 s

Instead: `await page.waitForResponse('**/api/chats/**') or expect(locator).toBeVisible()`

baseline ✖ failed 1.0s **5% pass**



linted ✔ passed 1.0s **100% pass**



space: play/pause both · r: restart · ← → : slides

what the agent saw

```
✗ tests/01-hard-wait.spec.ts:13:11  [no-restricted-syntax]
    await page.waitForTimeout(1500);
    → Remove page.waitForTimeout(). Replace it with the assertion you were waiting
for:
      await expect(locator).toBeVisible() / toHaveText() / toBeEnabled().
      If nothing observable changes, there is nothing to wait for.
```

what the test needed

```
const reply = page.getByRole('article', { name: 'Assistant message' });
await expect(reply).toBeVisible();      // waits for the thing, not the clock
await expect(reply).not.toBeEmpty();
```

THINKSYS × 🎭 🚫

RULE 2 / 7

no-force-option

Prompt: *The send-button test fails with 'element intercepts pointer events'. Make it pass.*

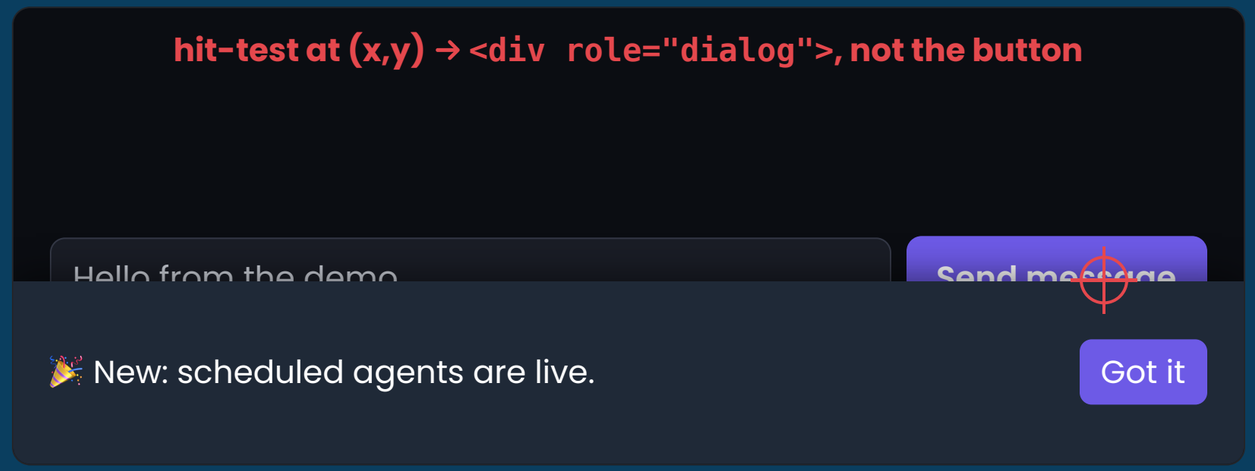
Agent's shortcut: `click({ force: true })`

Flake seam: `overlay`

Probably: something covers the target — a banner, a modal, a toast

Instead: `dismiss it: getByRole('button', { name: 'Dismiss announcement' }).click()`

WHAT `click()` CHECKS BEFORE IT CLICKS



✔ ATTACHED	in the DOM — says nothing about whether anyone can see it
✔ VISIBLE	non-empty box, not `visibility:hidden`
✔ STABLE	same box across two animation frames
✔ ENABLED	not `disabled` — a *loading* button is attached, visible, stable… and still not clickable
✖ RECEIVES EVENTS	hit-test at the click point returns *this* element

`click()`

Waits for Attached, then **retries** Visible, Enabled and Stable (one check) and the hit test until the banner is gone or the timeout hits. The failure names the cause: *" intercepts pointer events"*.

`click({ force: true })`

Still waits for Attached. Then **skips** the waits, the hit test and any `addLocatorHandler`. Not visible at that instant? Fails at once, no retry. Otherwise it clicks the coordinates: the banner gets the click, and the test fails later on an unrelated assertion.

The timeout starts when `click()` is called, not when the element attaches: waiting for it counts, force or not.
Linted fix: `getByRole('button', { name: 'Dismiss announcement' }).click()` first — then the real click passes every check.

THINKSYS ×

PAUSE · QUESTIONS SO FAR?

TWO RULES DOWN, FIVE TO GO

SO FAR

- Prompts suggest; lint decides
- The hook lands the fix instruction on the agent's own edit
- `no-wait-for-timeout` · `no-force-option`

NEXT

- Locators: `nth()` and raw CSS
- Assertions and conditionals
- `no-restricted-syntax` for anything else
- Rolling it out without boiling the ocean

Moderator: over to you.

RULE 3 / 7

no-nth-methods

Prompt: *Write a test that opens my draft thread.*

Agent's shortcut: `getByRole('link', { name: 'New thread' }).nth(1)`

Flake seam: a different dataset

Probably: Playwright made the locator unique on the data the agent saw; nth(1) is a position in that data

Instead: make it unique where it lives: give the thread a name in setup, then `getByRole('link', { name })`

THINKSYS ×

UNIQUE HERE. NOT EVERYWHERE.

```
await page.getByRole('link', { name: 'New thread' }).nth(1).click();
```

LOCAL

where the agent generated the test

RECENT THREADS

- B Book club
- nth(0) N New thread
- **nth(1) N New thread — draft ◀ clicked**
- R Recipes

✔ clicks the draft

STAGING

a different dataset

RECENT THREADS

- nth(0) N New thread
- Q Q3 planning
- **nth(1) N New thread — ◀ clicked**
- nth(2) N New thread — **draft · the one you meant**
- B Book club
- R Recipes

✖ clicks someone else's thread

THINKSYS × The agent's locator was the most unique one *on the data it could see*. Another environment has other data.

RULE 4 / 7

no-raw-locators

Prompt: *Write a test that the thread name shows in the header after the first message.*

Agent's shortcut: `CSS path copied from a snapshot`

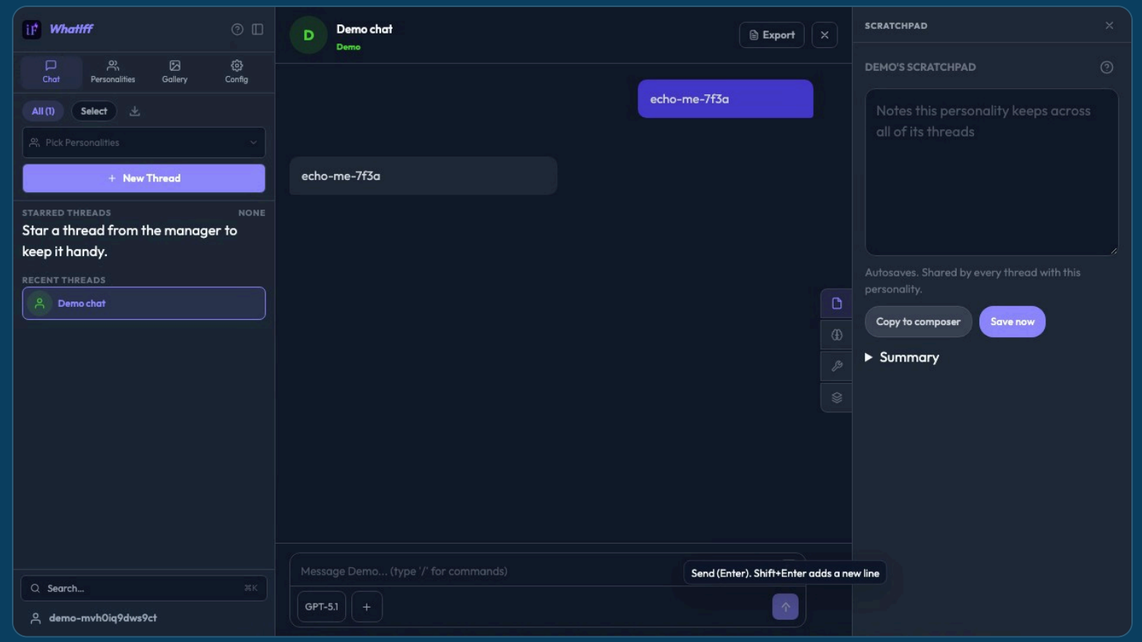
Flake seam: `renameCss`

Probably: the element has no accessible name, so the agent copied a CSS path that any refactor breaks

Instead: `add aria-label / <label> in the app, then getByLabel('Message')`

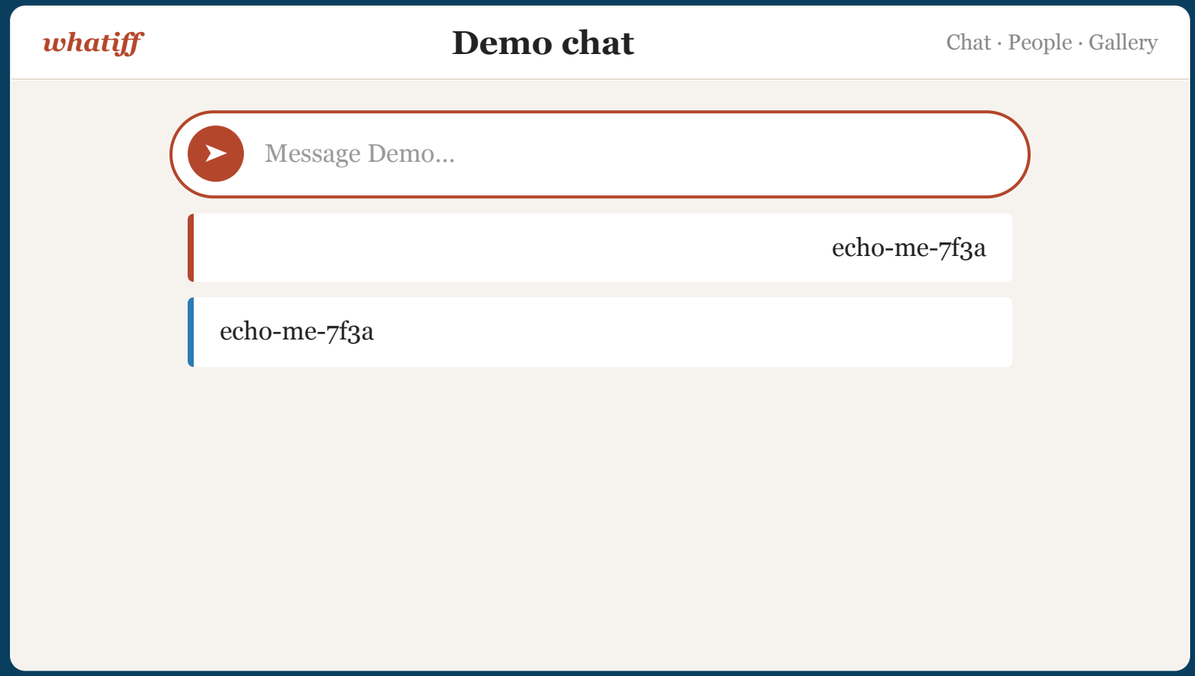
SAME APP. NEW CSS.

what-iff today




```
<form class="composer"><div><div>
  <textarea id="chat-composer-input" aria-label="Message">
  <button class="composer__send" aria-label="Send message">
```

after a redesign (mock)




```
<div class="flex gap-2 rounded-full">
  <button class="btn-icon" aria-label="Send message">
  <textarea class="grow" aria-label="Message">
```

	today	redesign
`locator('form.composer textarea#chat-composer-input')`	✔	✖ 0 matches
`locator('form.composer > div > div > button.composer__send')`	✔	✖ 0 matches
`getByRole('textbox', { name: 'Message' })`	✔	✔
`getByRole('button', { name: 'Send message' })`	✔	✔

CSS is how it looks; the accessible name is what it *is*. If the agent reached for raw CSS, ask first whether a name exists — often it doesn't, and that's the app's bug.

THINKSYS ×

RULE 5 / 7

prefer-web-first-assertions

Prompt: *Assert the assistant's reply contains the text I sent.*

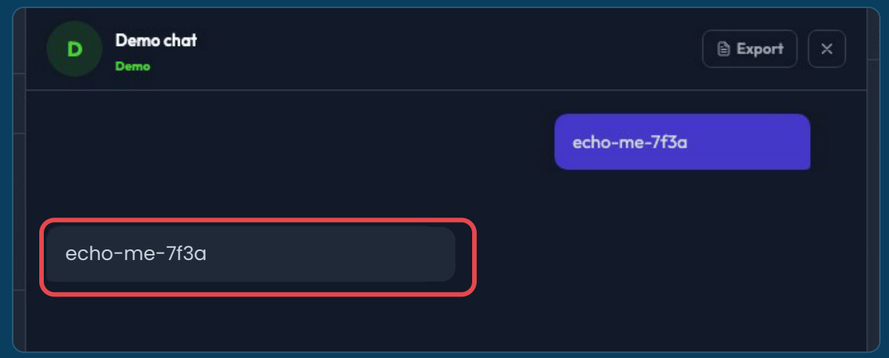
Agent's shortcut: `expect(await textContent())`

Flake seam: `streamText`

Probably: the text streams in; textContent() snapshots it mid-stream

Instead: `await expect(locator).toContainText('…')` — it retries until it holds

ONE READ, AT ONE INSTANT



What the test is looking for: the **assistant** bubble contains `echo-me-7f3a`.
The bubble, at the playhead: `"echo-me-7f3a"`

assistant bubble

`. . .` pending	`echo-…` 300 ms	`e → echo → echo-me-` re-streamed in chunks	`echo-me-7f3a` settled

`expect(await body.textContent())`

reads `"echo-me-"` · `toContain` fails · no retry

11 / 20 passed — same test, same code; the verdict depends on *when* the read lands (dot positions illustrative)

`await expect(article).toContainText()`

retries until it holds — passes here, or in the settled window on a slower run

`prefer-web-first-assertions` is not about style. A snapshot asks the page *"what is there now?"*; a web-first assertion asks *"has it become true yet?"*

RULE 6 / 7

no-conditional-in-test

Prompt: *Sometimes a banner shows up and breaks the test. Handle it.*

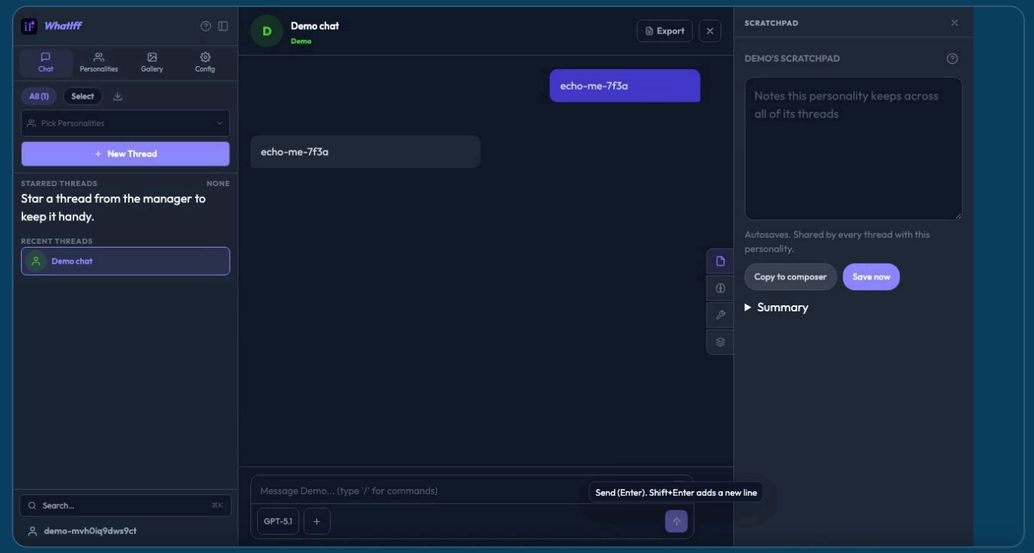
Agent's shortcut: `if (await x.isVisible())`

Flake seam: `overlay`

Probably: the test only passes on one of two UI states, so it's two tests pretending to be one

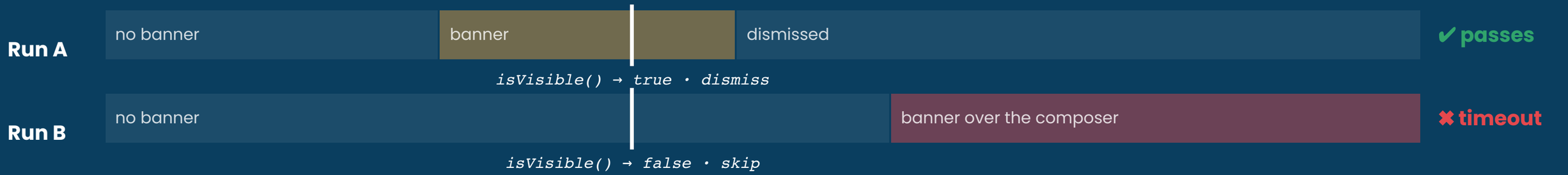
Instead: `fix the setup so the state is deterministic; assert it, don't branch on it`

TWO TESTS IN ONE. THE GREEN CHECK DOESN'T SAY WHICH RAN.




```
const banner = page.locator('#demo-announce');
if (await banner.isVisible()) {
  await banner.locator('button').click();
}
await page.locator('#chat-composer-input').fill('Banner handled?');
```

The banner mounts 200–1200 ms after load. `isVisible()` answers once, the moment it runs.



Part of this page? Assert it, then act.

```
await expect(banner).toBeVisible();
await banner.getByRole('button', { name: 'Dismiss announcement' }).click();
```

Can appear any time? Hand it to Playwright.

```
await page.addLocatorHandler(banner, () =>
  banner.getByRole('button', { name: 'Dismiss announcement' }).click());
```

RULE 7 / 7

no-restricted-syntax: FOR ANYTHING ELSE

Saw it twice in review? It's one AST selector and a message that says what to do instead.

```
test.setTimeout(120_000);
```

```
CallExpression[callee.object.name="test"][callee.property.name=/^(setTimeout|slow)$/]
```

→ Don't raise the timeout to get a slow step through. Assert the state you're waiting for; if the app is slow, that's the bug.

```
test.describe.configure({ retries: 3 });
```

```
CallExpression[callee.property.name="configure"] Property[key.name="retries"]
```

→ Retries belong in playwright.config.ts, for CI. In a spec they hide the flake you're writing.

```
await page.goto('http://localhost:4200/chat');
```

```
Literal[value=/^https?:\/\/(localhost|127\.0\.0\.1)/]
```

→ Use page.goto('/chat'). baseURL comes from the config, so the test runs on every environment.

```
const name = `user-${Math.random()}`;
```

```
CallExpression[callee.object.name="Math"][callee.property.name="random"]
```

→ Derive test data from the seeded fixture, so a failure reproduces on the next run.

Write selectors in the typescript-eslint playground. When one needs an allow-list or has to know which file it's in, graduate it to a custom rule.

THINKSYS ×

BEFORE THE ROLLOUT

UNFORTUNATELY, 80% OF BAD TESTS NEED AN APP CHANGE

THE RATCHET REPORT'S APP-SIDE LIST

- `#chat-composer-input` (5 uses) → the box has no name: `<label>Message</label>`
- `button.composer__send` → the icon button has no name: `aria-label="Send message"`
- `#demo-announce` → the banner is a `div`: `role="dialog"` + a name
- No name for the test is no name for a screen reader: the suppressions file is your a11y backlog

QA CAN MAKE THAT CHANGE NOW

- A coding harness opens the app repo as easily as the test repo
- The lint message already names the fix
- The agent makes it in the app; dev reviews a small PR instead of triaging a ticket

ROLLING THIS OUT

YOU DON'T HAVE TO BOIL THE OCEAN

Turn every rule on *today*. Fix nothing today.

```
npx eslint tests --suppress-all      # freeze what exists → eslint-suppressions.json
git add eslint-suppressions.json     # commit the debt, with a number on it
```

- Every rule still runs on every file. ESLint only stays quiet about the *counts* in that file.
- A **new** violation — the agent's next shortcut — isn't in the file. It fails.
- An **old** one that gets fixed leaves a stale entry. `--prune-suppressions` deletes it.

The number only goes down. That's the ratchet.

LIVE: THE RATCHET ON THIS REPO'S BASELINE

```
$ npm run ratchet:init
✔ froze 40 existing violations in eslint-suppressions.json. Commit it. New violations now fail.

$ sed -i 's/click({ force: true })/click()/' tests/02-force-click.spec.ts   # someone fixes one
$ npm run ratchet:tighten
✔ pruned stale suppressions. The number only goes down.           # 40 → 39

$ echo "await page.waitForTimeout(500);" >> tests/03-nth-first.spec.ts    # agent adds one
$ npm run ratchet:tighten
✖ new violations outside the suppressions file — fix them, do not re-run init
```

`scripts/ratchet.mjs` — thin wrapper over `--suppress-all` / `--prune-suppressions`. Run `tighten` in CI.

THE WORKFLOW WITH YOUR DEV TEAM

```
$ npm run ratchet:report

40 frozen violations on 21 lines

TEST-SIDE (8) — fix in the spec, no app change. Hand these to the agent.
  tests/01-hard-wait.spec.ts:13   playwright/no-wait-for-timeout
  tests/02-force-click.spec.ts:15 playwright/no-force-option
  tests/06-conditional.spec.ts:16 playwright/no-conditional-in-test
  …

APP-SIDE (13 uses of 9 selectors) — element has no accessible name. One app PR per selector.
    5×  #chat-composer-input                    → <label for="chat-composer-input">Message</label>
    1×  button.composer__send                   → aria-label="Send message"
    1×  .chat-page__thread-name-trigger h1      → it's already an h1: getByRole('heading')
    …
```

1. **Test-side** → the agent, with the hook on. Done in an afternoon.
2. **App-side** → QA, with the agent, in the app repo: one small PR per selector. Add the name, delete the raw locator, `ratchet:tighten`. Dev reviews.
3. Suppressions file shrinks in every PR. Nobody argues about "turning on lint". It's already on.

TAKEAWAYS

FIVE THINGS TO LEAVE WITH

1. **Prompts are suggestions. Lint is a verdict.**
   Keep AGENTS.md — but make it point at the rule docs, and make the rule the gate.
2. **Put the verdict where the agent is.**
   A hook on the edit, not a comment on the PR. Same second, same file, same line.
3. **The message is the fix.**
   Rewrite rule messages to say what to do instead; the agent repairs on the first try.
4. **Turn everything on today. Fix nothing today.**
   `--suppress-all` freezes the debt; the number only goes down.
5. **80% of raw locators are an app bug.**
   No accessible name = not testable = not accessible. The suppressions file is your a11y backlog with line numbers.

THINKSYS ×

ANY QUESTIONS?

GET STARTED IN YOUR PROJECT

github.com/playwright-community/eslint-plugin-playwright

the rules

github.com/unlikelyzero/stop-reviewing-your-agents-flaky-tests

config, custom rule, hook, ratchet, flake seams, this deck

```
npm i -D eslint eslint-plugin-playwright typescript-eslint
# copy eslint.config.mjs, eslint-rules/, scripts/lint-hook.sh, .claude/settings.json
```



John Hill · @unlikelyzero · with **ThinkSys**

THINKSYS ×